Manual de Laboratorio: Diseño de Syscalls

Laboratorio de Sistemas Operativos
Facultad de Ciencias, UNAM

1. Objetivos de la Práctica

1. **Aislamiento de Privilegios:** Implementar una barrera de protección estricta entre el Modo Usuario (no privilegiado) y el Modo Kernel (privilegiado).
2. **Diseño de una ABI:** Comprender y programar la interfaz binaria de aplicaciones (*Application Binary Interface*) mediante la instrucción de excepción `SVC`.
3. **Gestión de Hardware Segura:** Desarrollar un Kernel capaz de validar peticiones de usuario antes de manipular registros físicos del hardware.

2. Requisitos y Material

- Raspberry Pi Pico (RP2040).
- Cable Micro-USB de datos.
- Protoboard, 1 LED, 1 Botón y resistencias de 330Ω y $10k\Omega$.
- Entorno de desarrollo configurado (`pico-sdk`, `CMake`, `arm-none-eabi-gcc`).

3. Ejercicio: Implementación de Syscalls para Control de GPIO

En el desarrollo de sistemas operativos modernos, permitir que una aplicación de usuario acceda directamente a la memoria de periféricos (registros SIO/GPIO en `0xD0000000`) es una vulnerabilidad crítica de seguridad.

En esta práctica, trabajaremos sobre **PicoOS**. El reto consiste en que el código de la aplicación (`user_app.c`) **no tiene permiso** para incluir librerías de hardware ni punteros directos. Si la aplicación desea encender un LED o leer un botón, debe realizar una "Llamada al Sistema"(*Syscall*), cediendo el control al procesador para que el Kernel verifique y ejecute la acción en su nombre.

3.1. Especificaciones técnicas

La comunicación entre el usuario y el kernel se rige por las siguientes reglas basadas en el estándar ARM AAPCS:

- **Instrucción de Entrada:** Se utiliza `svc #0` para generar una excepción de software.
- **Registro de Control ($r7$):** El identificador de la syscall (ID) debe cargarse en el registro $r7$ antes de la llamada.
- **Paso de Parámetros:** Los argumentos se envían en los registros $r0, r1, r2$ y $r3$.
- **Retorno de Datos:** El Kernel deposita el resultado de la operación en el registro $r0$ del usuario.

4. Estructura del Proyecto

El sistema se compone de archivos de Kernel (Privilegiados) y archivos de Usuario (No Privilegiados).

4.1. Componentes del Kernel

- **`main.c`:** Configura el manejador de excepciones y lanza la aplicación de usuario.
- **`svc_handler.S`:** Wrapper en ensamblador que extrae el Stack Pointer y el ID de syscall para pasarlos a C.
- **`kernel_service.c`:** El *Dispatcher*. Contiene la lógica para decidir qué driver llamar según el ID en $r7$.
- **`kernel_drivers.c`:** El único archivo con acceso real a los registros del RP2040.

4.2. Componentes de Usuario

- **`syscalls.S`:** Librería de interfaz que oculta la complejidad del ensamblador tras funciones de C como `sys_gpio_set`.
- **`user_app.c`:** Lógica de la aplicación escrita por el alumno.

5. Tareas a Realizar

5.1. Tareas Específicas

1. Completar los fragmentos marcados con `TODO`.
2. Generar un diagrama de flujo que muestre el recorrido completo desde la invocación de una syscall hasta el retorno de control al usuario.
3. Anexar capturas de pantalla de cada sección `TODO` completada con código funcional.
4. Implementar la Fase B: Sistema de toggle con detección de flanco y debounce en `user_app.c`.
5. Grabar un video muy breve (15 a 30 segundos por fase) de demostración del funcionamiento de ambas fases en hardware real.

5.2. Dearrollo

Fase A: Los alumnos deberán completar los fragmentos de código marcados con `TODO` en los archivos proporcionados. Con esto, tendrán un puente simple que copia el estado del botón al LED usando `sys_gpio_get` y `sys_gpio_set`.

Fase B: Sistema Bi-estable con Debounce. Una vez validada la Fase A, el alumno debe reprogramar `user_app.c` para implementar un sistema de iluminación inteligente:

1. **Detección de Flanco:** El LED debe cambiar de estado (*toggle*) únicamente cuando el botón sea presionado y soltado (detección de flanco de bajada o *falling edge*).
2. **Software Debounce:** Debe implementar una lógica de retardo o verificación de estado para evitar falsos disparos causados por el ruido mecánico del botón.
3. **Restricción:** Está estrictamente prohibido usar punteros o la dirección `0xD0000000` dentro de este archivo. Solo se permiten syscalls.

6. Criterios de Evaluación

Deberá incluir el diagrama de flujo de la fase A. Además, se deben anexar capturas de pantalla de todas las secciones `TODO` completadas.

- **Punto Extra (+1.0):** Si el reporte completo está realizado en LaTeX utilizando el paquete `listings` para mostrar el código de forma profesional.
- **30 % - Actividad Base (Fase A):** El LED refleja el estado del botón en tiempo real a través de syscalls.
- **30 % - Reto Bi-estable:** Implementación correcta del *toggle* y el *debounce* en espacio de usuario.
- **40 % - Reporte y Calidad de Código:**
  - Explicación detallada del flujo de una excepción desde que se ejecuta `svc` hasta que regresa al usuario.
  - Comentarios técnicos en las secciones completadas de los drivers y la aplicación.
  - Enlace al video de demostración de la placa funcionando para las fases A y B. **Solo se evaluarán las secciones que se demuestren claramente en el video.**

6.1. Validación en Video

Solo para alumnos que NO validaron la práctica presencialmente en clase. Deberán entregar un video tutorial explicando el funcionamiento. **No se aceptarán videos con audio poco claro o mala calidad de imagen que impida leer el código o ver los componentes.**

Rúbrica del Video:

1. **Explicación del Programa (80 %):** Se divide en 4 etapas:
   - Explicación del flujo de la Syscall usando su diagrama adaptado a la Fase B (50 %).
   - Lógica de lectura de botón y manejo de estados (10 %).
   - Implementación del Software Debounce (10 %).
   - Lógica de detección de flanco de bajada y Toggle (10 %).
2. **Demostración Física (5 %):** Muestra de la placa funcionando con el reto de la Fase B.
3. **Proceso de Compilación (5 %):** Explicación completa del proceso de compilación y carga a la placa.
4. **Calidad de Edición y Claridad (10 %):** Estructura del video, fluidez y claridad en la voz.